Control Hazards

- Also called as branch hazard,An occurrence in which the proper instruction can not execute in the proper clock cycle because the instruction that was fetched is not the one that is needed, i.e the flow of instruction addresses is not what pipeline expected.

Suppose our laundry crew was given the happy task of cleaning the uniforms of a football team. Given how filthy the laundry is, we need to determine whether the detergent and water temperature setting we select is strong enough to get the uniforms clean but not so strong that the uniforms wear out sooner. In our laundry pipeline, we have to wait until the second stage to examine the dry uniform to see if we need to change the washer setup or not. What to do?

Here is the first of two solutions to control hazards in the laundry room and its computer equivalent.

Stall: Just operate sequentially until the first batch is dry and then repeat until you have the right formula. This conservative option certainly works, but it is slow.

The equivalent decision task in a computer is the branch instruction. Notice that we must begin fetching the instruction following the branch on the very next clock cycle. But, the pipeline cannot possibly know what the next instruction should be, since it *only just received* the branch instruction from memory! Just as with laundry, one possible solution is to stall immediately after we fetch a branch, waiting until the pipeline determines the outcome of the branch and knows what instruction address to fetch from.

Performance of “stall on branch”

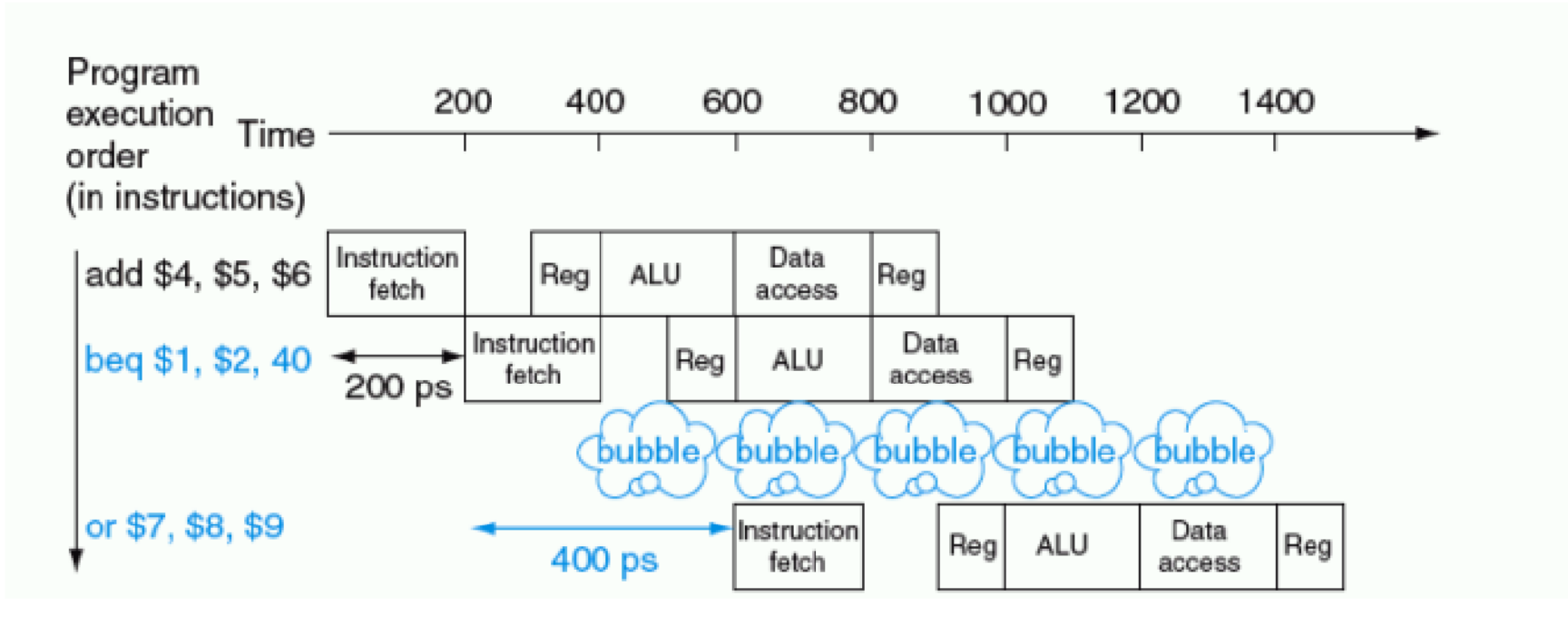


FIGURE 6.7 Pipeline showing stalling on every conditional branch as solution to control hazards. There is a one-stage pipeline stall, or bubble, after the branch. In reality, the process of creating a stall is slightly more complicated, as we will see in Section 6.6. The effect on performance, however, is the same as would occur if a bubble were inserted.

- Let us assume that we put in enough extra h/w so that we can test registers, calculate the branch address, and update the PC during the second stage of the pipeline. Even with this extra h/w the pipeline involving conditional branches would look like the fig in previous slide.
- Estimate the impact on the clock cycles per instruction(CPI) of stalling on branches. Assume all other instructions have a CPI 1.
- It has been found that branches are 13% of the instructions executed in SPECint2000,since other instructions run have a CPI of 1 and branches took one extra clock cycle for the stall,hence the CPI 1.13,slowdown of 1.13 versus the ideal case.jumps also incur stalls.

Core MIPS	Name	Integer	Fl. pt.
add	add	0%	0%
add immediate	addi	0%	0%
add unsigned	addu	7%	21%
add immediate unsigned	addiu	12%	2%
subtract unsigned	subu	3%	2%
and	and	1%	0%
and immediate	andi	3%	0%
or	or	7%	2%
or immediate	ori	2%	0%
nor	nor	3%	1%
shift left logical	sll	1%	1%
shift right logical	srl	0%	0%
load upper immediate	lui	2%	5%
load word	lw	24%	15%
store word	sw	9%	2%
load byte	lbu	1%	0%
store byte	sb	1%	0%
branch on equal (zero)	beq	6%	2%
branch on not equal (zero)	bne	5%	1%
jump and link	jal	1%	0%
jump register	jr	1%	0%
set less than	slt	2%	0%
set less than immediate	slti	1%	0%
set less than unsigned	sltu	1%	0%

Arithmetic core + MIPS-32	Name	Integer	Fl. pt.
FP add double	add.d	0%	8%
FP subtract double	sub.d	0%	3%
FP multiply double	mul.d	0%	8%
FP divide double	div.d	0%	0%
load word to FP double	l.d	0%	15%
store word to FP double	s.d	0%	7%
shift right arithmetic	sra	1%	0%
load half	lhu	1%	0%
branch less than zero	bltz	1%	0%
branch greater or equal zero	bgez	1%	0%
branch less or equal zero	blez	0%	1%
multiply	mul	0%	1%

- If we cannot resolve the branch in the second stage,as is often the case for longer pipelines, then we would see larger slowdown if we stall on branches.
- The cost of this option is too high for most computers to use and motivates a second solution to the control hazard.

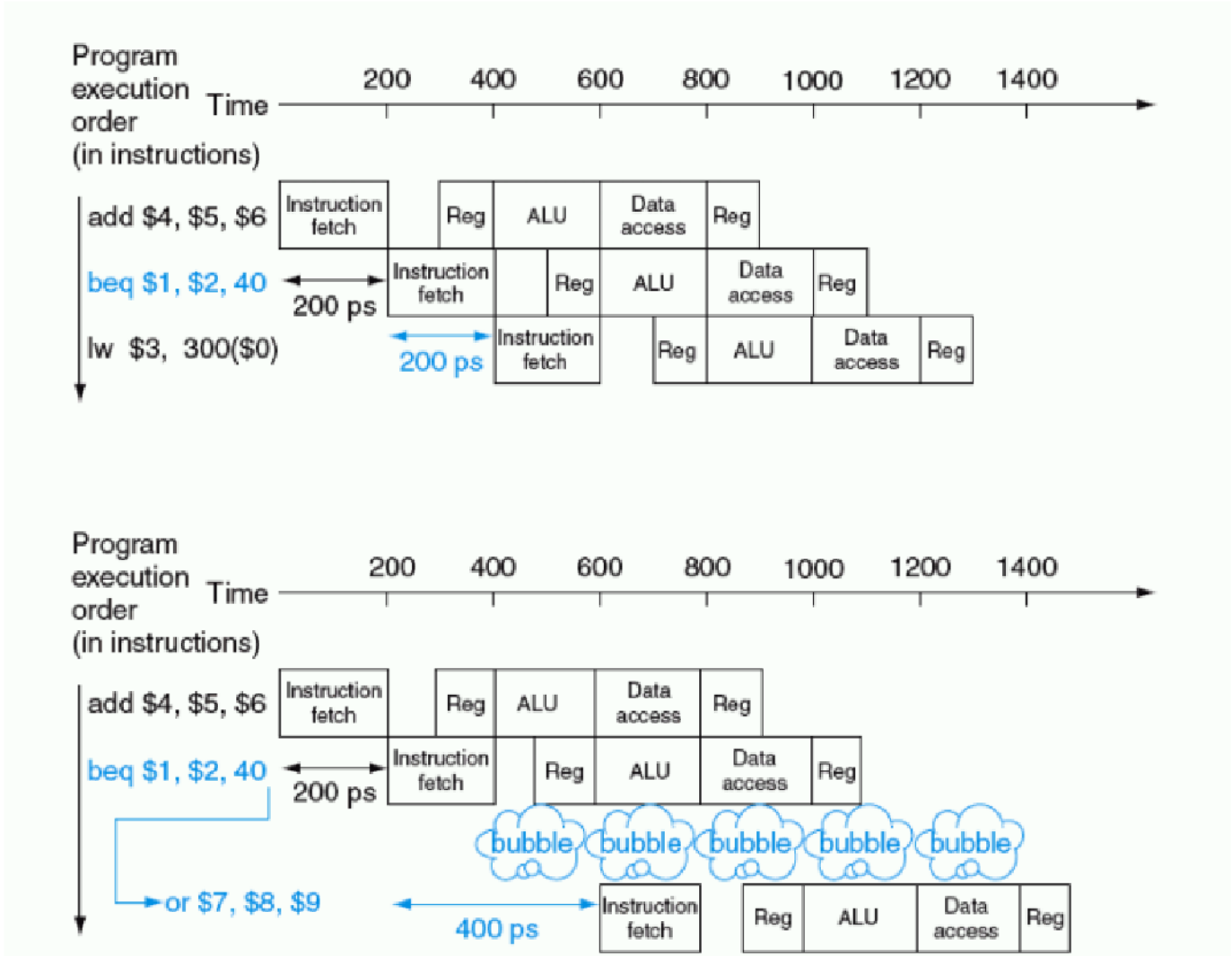
Predict:*with reference to laundry example,if you are pretty sure of having the right formula to wash uniforms,then just predict that it will work and wash the second load while waiting for the first load to dry.*

This option does not slowdown the pipeline when you are correct.

When you are wrong ,then you need to redo the load that was washed while guessing the decision.

- Computers do indeed use prediction to handle branches.One simple approach is to always predict that the branches will be ***untaken***.
- When you are right,the pipeline proceeds at full speed.Only when branches are taken does the pipeline stall.
-

Predicting that branches are not taken as a solution to control hazard



Branch prediction

- Some branches predicted as taken and some untaken.In laundry example,the dark or home uniforms might take one formula ,while the light or road uniforms might take another.
- As a computer example,at the bottom of loops are branches that jump back to the top of the loop.Since they are likely to be taken and they branch backwards,we could always predict taken for branches that jump to an earlier address.
- Dynamic h/w predictors make their guesses depending on the behaviour of each branch and may change predictions for a branch over the life fo a program.
- In our analogy, a dynamic prediction, a person would look at how dirty the uniform was and guess at the formula, adjusting the next guess depending on the success of recent guesses

- One popular approach to dynamic prediction of branches is keeping a history for each branch as taken or untaken, and then using the recent past behaviour to predict the future.
- Survey says, dynamic branch predictors can correctly predict branches with over 90% accuracy.
- When the guess is wrong, the pipeline control must ensure that the instructions following the wrongly guessed branch have no effect and must restart the pipeline from the proper branch address.
- In laundry analogy, we must stop taking new loads so that we can restart the load that we incorrectly predicted.

- Third approach to Control hazard: *delayed decision*

In our analogy, whenever you are going to make such a decision about laundry, just place a load of nonfootball clothes in the washer while waiting for football uniforms to dry. As long as you have enough dirty clothes that are not affected by the test, this solution works fine.

- *The delayed branch always execute the next sequential instruction with the branch taking place after that one instruction delay.*
- *MIPS s/w will place an instruction immediately after the delayed branch instruction that is not affected by branch and a taken branch changes the address of the instruction that follows this safe instruction*

- The add instruction before the branch in Fig. does not affect the branch and can be moved after the branch to fully hide

the branch delay. Since delayed branches are useful when the branches are short, no processor uses a delayed branch of more than 1 cycle. For longer branch delays, hardware-based branch prediction is usually used.

A pipeline Datapath

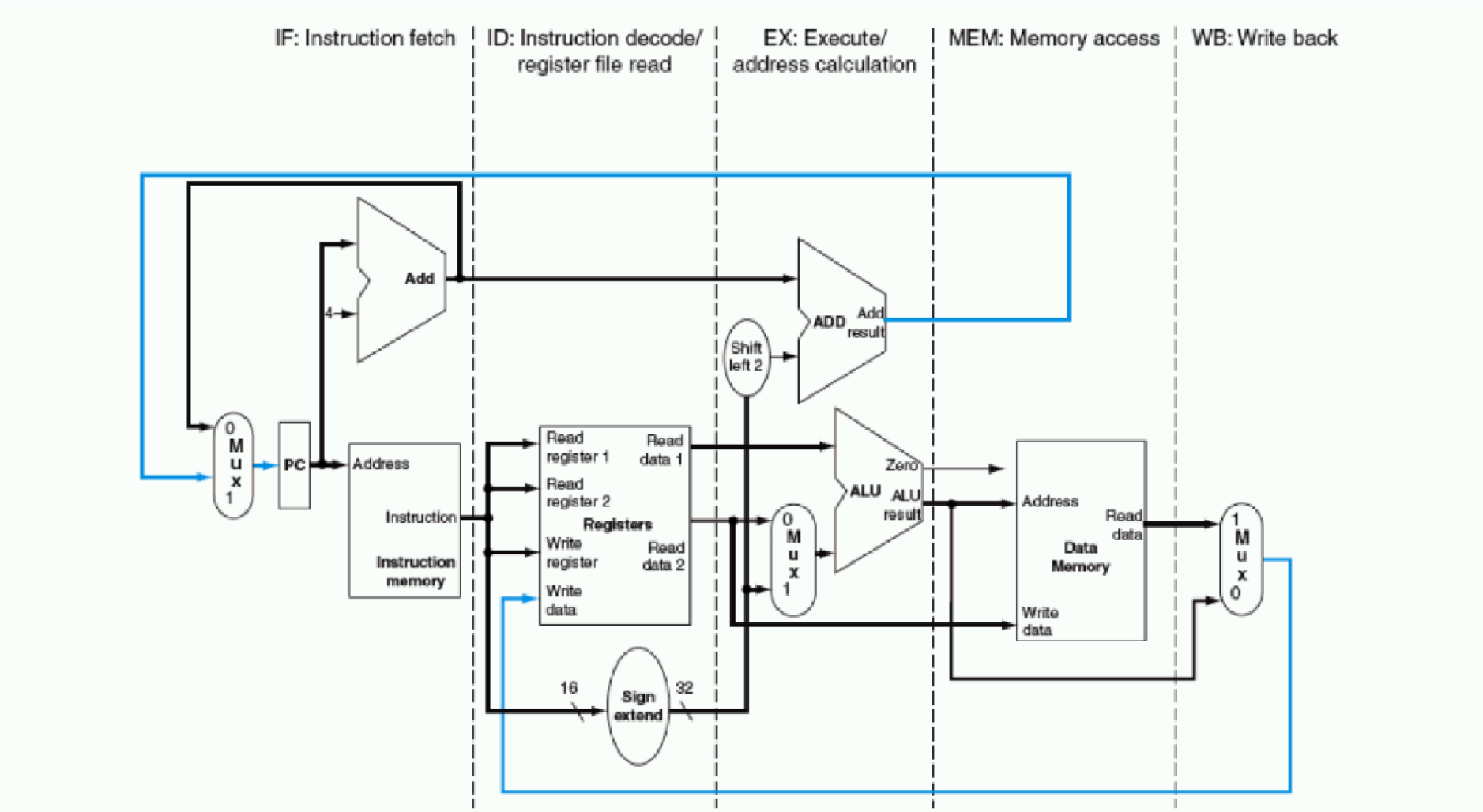


FIGURE 6.9 The single-cycle datapath from Chapter 5 (similar to Figure 5.17 on page 307). Each step of the instruction can be mapped onto the datapath from left to right. The only exceptions are the update of the PC and the write-back step, shown in color, which sends either the ALU result or the data from memory to the left to be written into the register file. (Normally we use color lines for control, but these are data lines.)

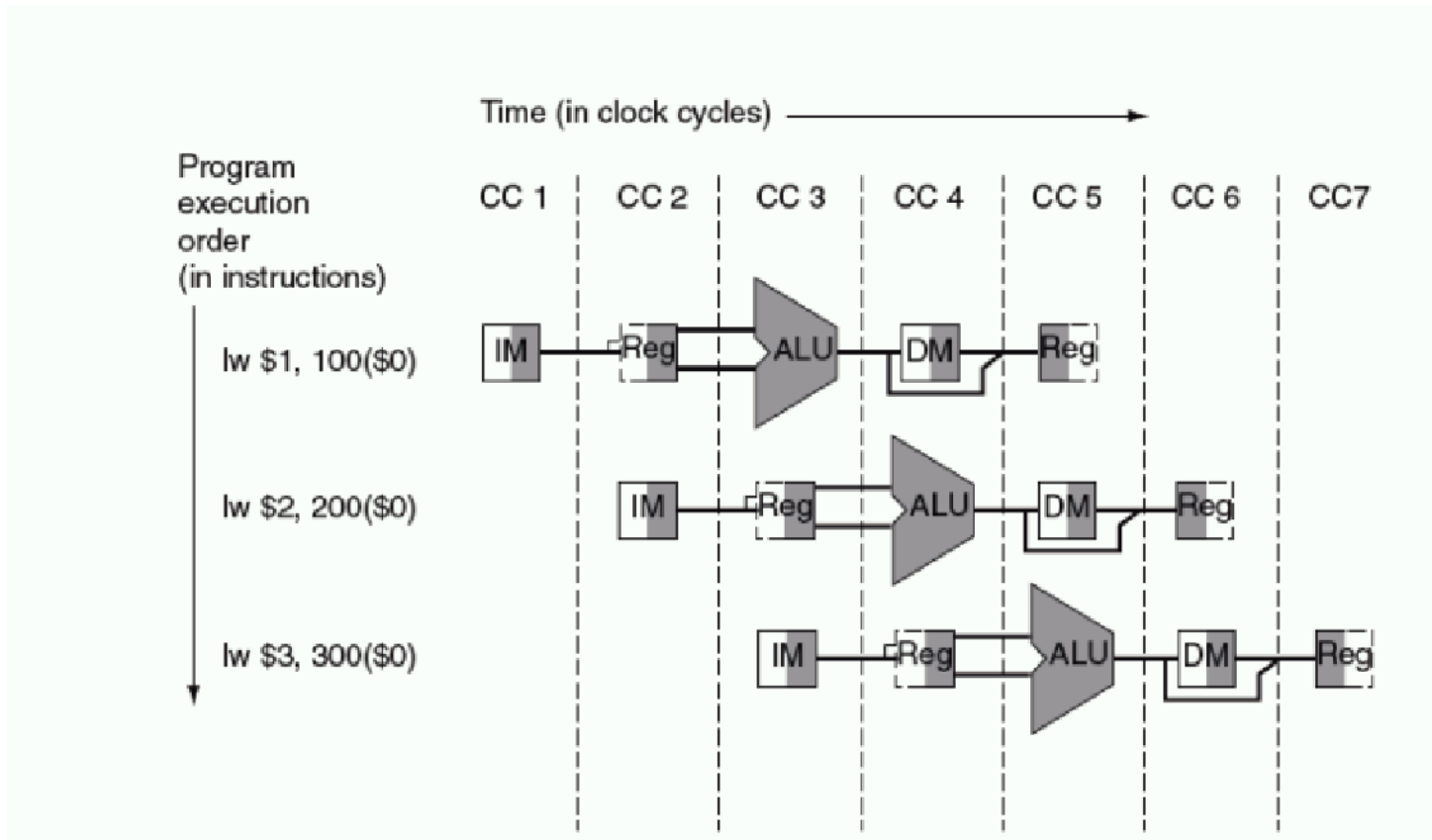

Time (in clock cycles)
Program execution order (in instructions)
CC 1
CC 2
CC 3
CC 4
CC 5
CC 6
CC7
lw $1, 100($0)
lw $2, 200($0)
lw $3, 300($0)
IM
Reg
ALU
DM
Reg

FIGURE 6.11 The pipelined version of the datapath in Figure 6.9. The pipeline registers, in color, separate each pipeline stage. They are labeled by the stages that they separate; for example, the first is labeled *IF/ID* because it separates the instruction fetch and instruction decode stages. The registers must be wide enough to store all the data corresponding to the lines that go through them. For example, the IF/ID register must be 64 bits wide because it must hold both the 32-bit instruction fetched from memory and the incremented 32-bit PC address. We will expand these registers over the course of this chapter, but for now the other three pipeline registers contain 128, 97, and 64 bits, respectively.

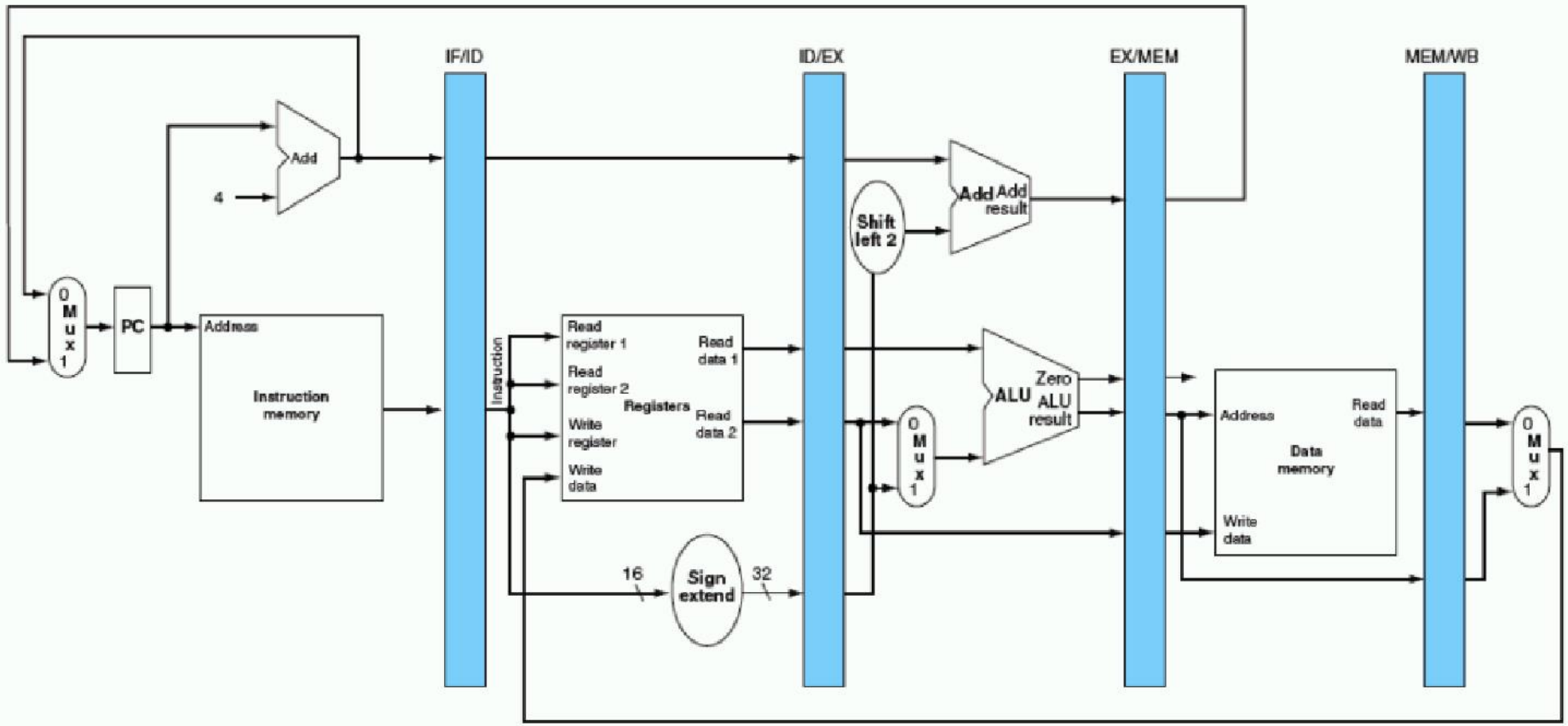
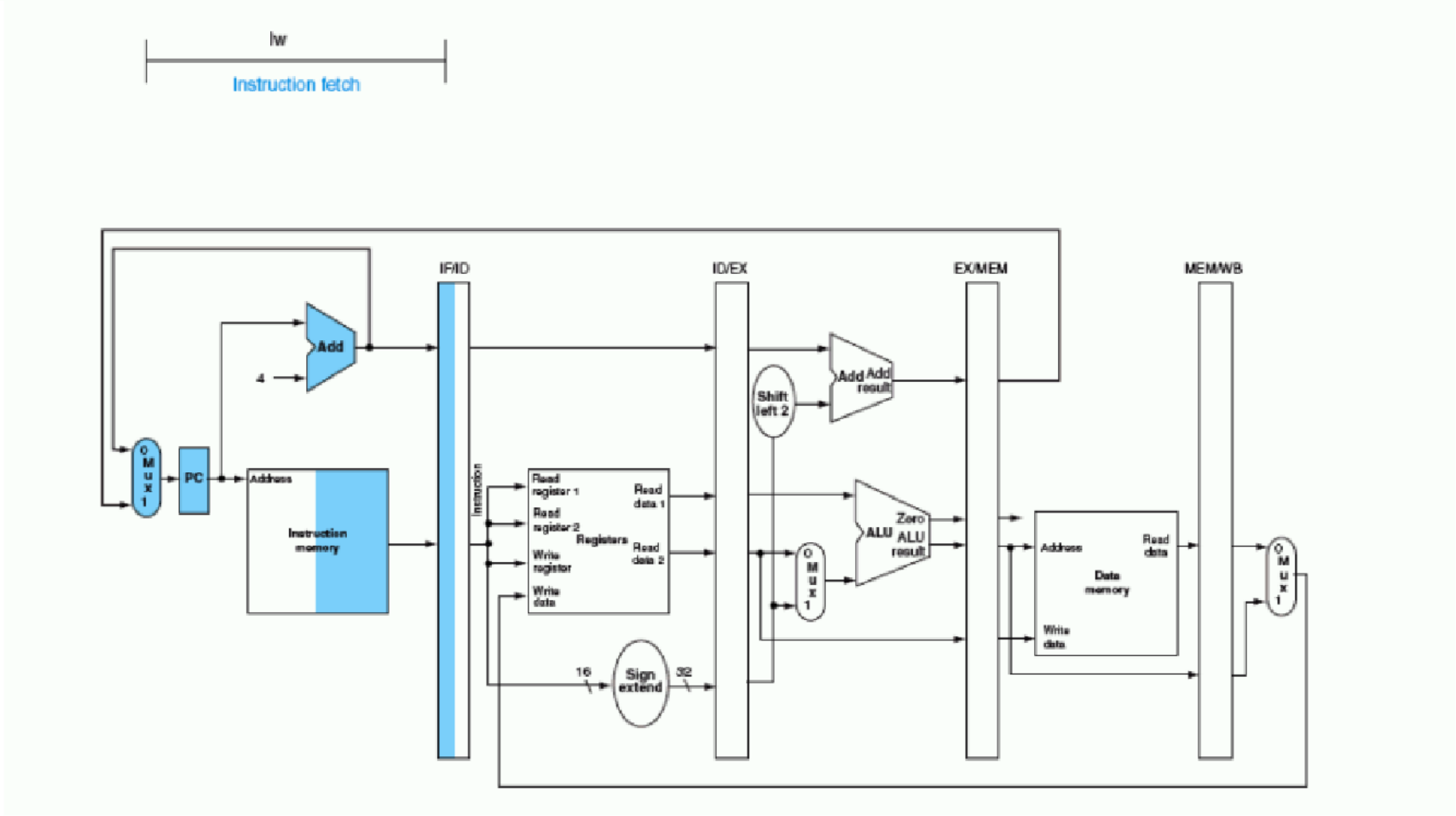


FIGURE 6.12 IF and ID: first and second pipe stages of an instruction, with the active portions of the datapath in Figure 6.11 highlighted. The highlighting convention is the same as that used in Figure 6.4. As in Chapter 5, there is no confusion when reading and writing registers because the contents change only on the clock edge. Although the load needs only the top register in stage 2, the processor doesn't know what instruction is being decoded, so it sign-extends the 16-bit constant and reads both registers into the ID/EX pipeline register. We don't need all three operands, but it simplifies control to keep all three.



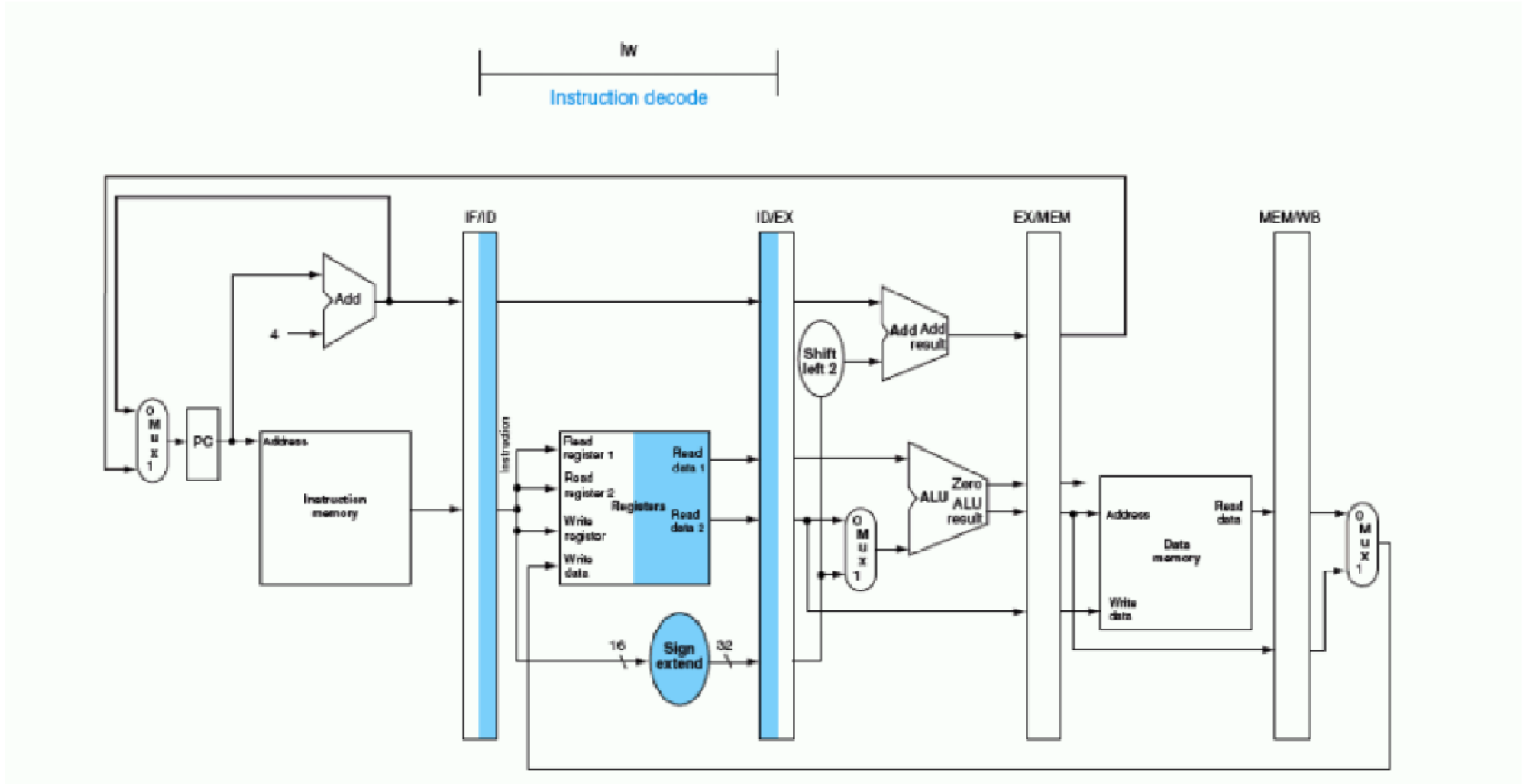
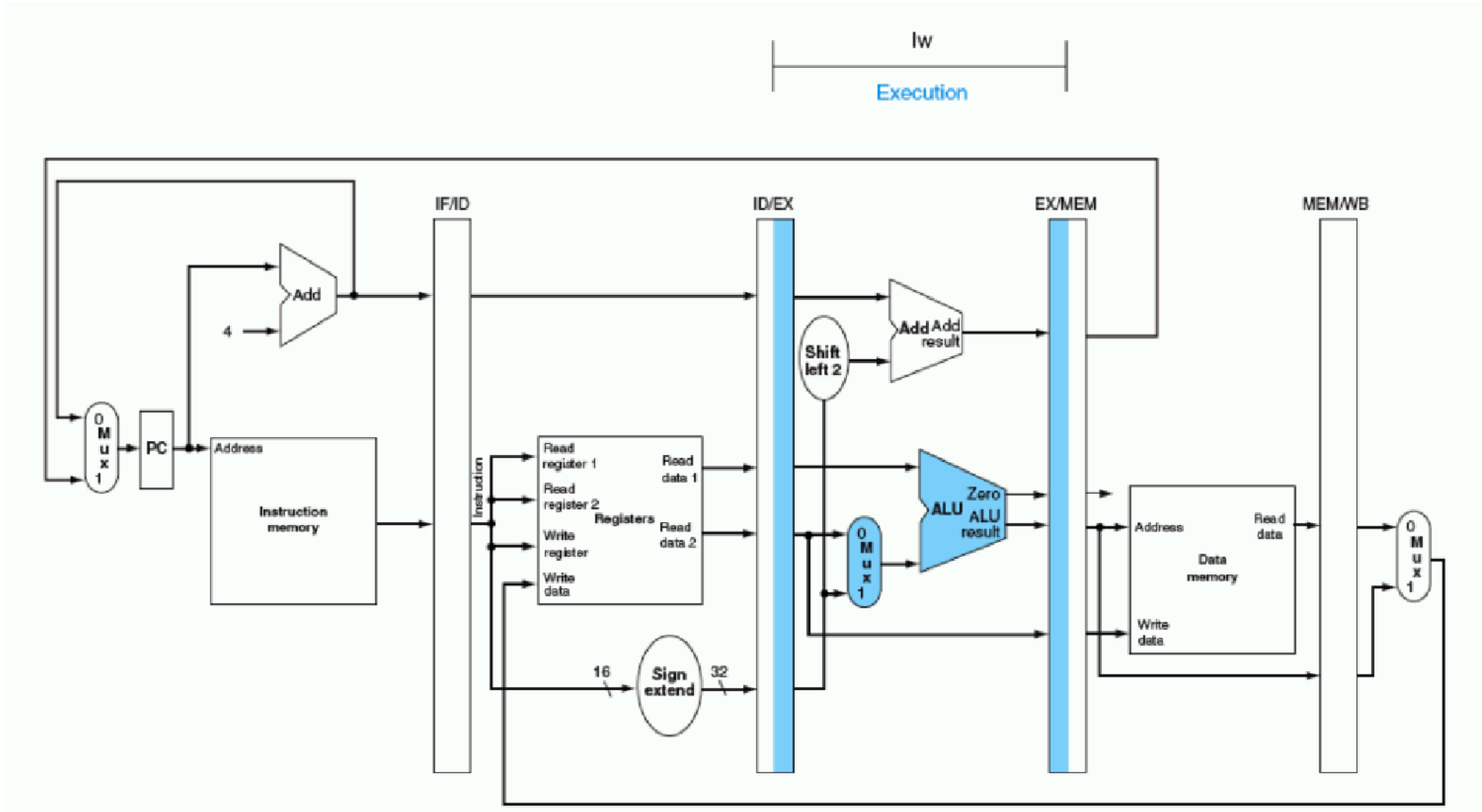

lw
Instruction decode
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M u x
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Sign extend
16
32
Shift left 2
Add
Add result
ALU
Zero
ALU result
Address
Data memory
Read data
Write data

FIGURE 6.13 EX: the third pipe stage of a load instruction, highlighting the portions of the datapath in Figure 6.11 used in this pipe stage. The register is added to the sign-extended immediate, and the sum is placed in the EX/MEM pipeline register.



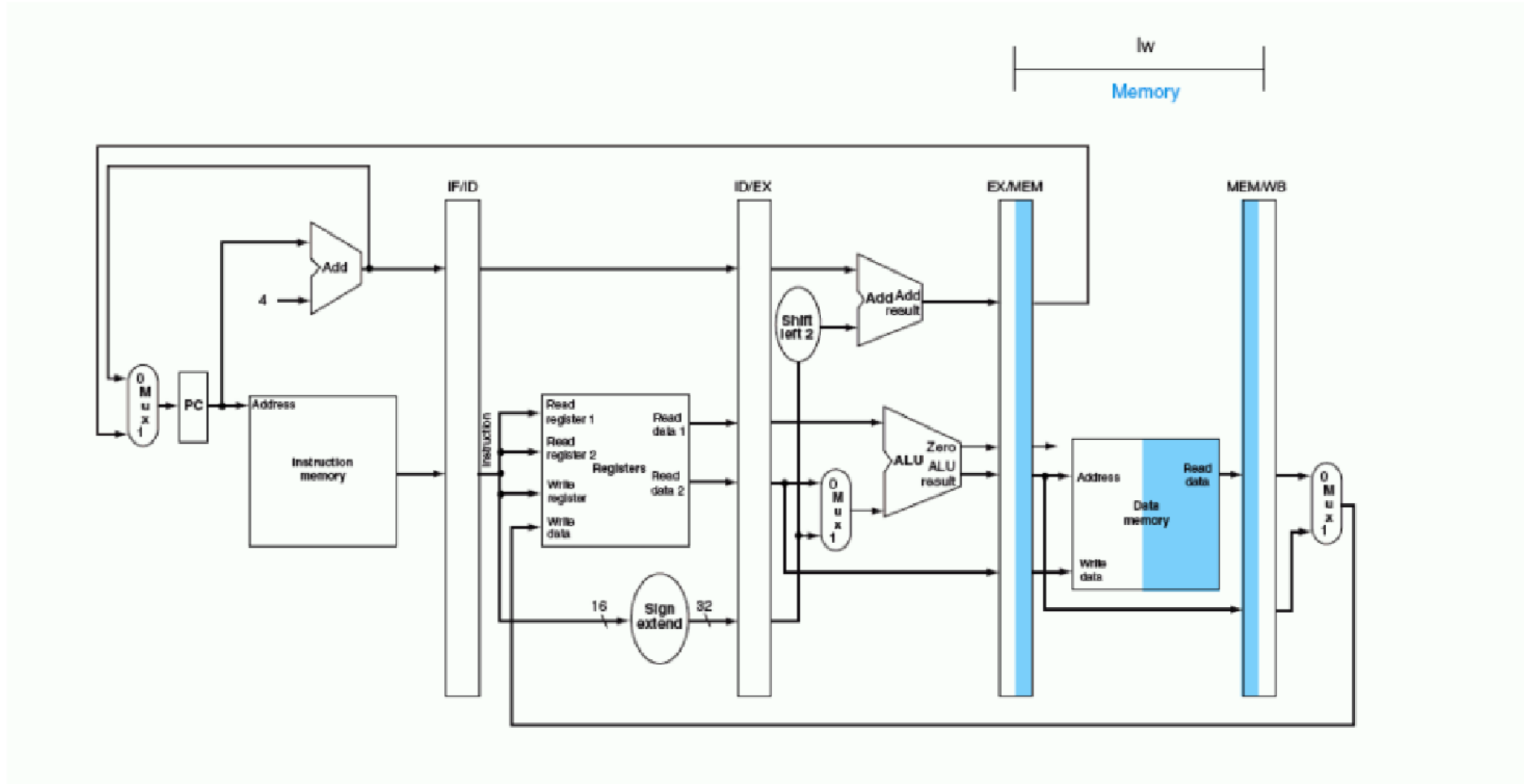

lw
Memory
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M
u
x
0
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Shift left 2
Add
Add result
ALU
Zero
ALU result
Address
Data memory
Read data
Write data
16
Sign extend
32

FIGURE 6.16 MEM and WB: the fourth and fifth pipe stage of a store instruction. In the fourth stage, the data is written into data memory for the store. Note that the data comes from the EX/MEM pipeline register and that nothing is changed in the MEM/WB pipeline register. Once the data is written in memory, there is nothing left for the store instruction to do, so nothing happens in stage 5.

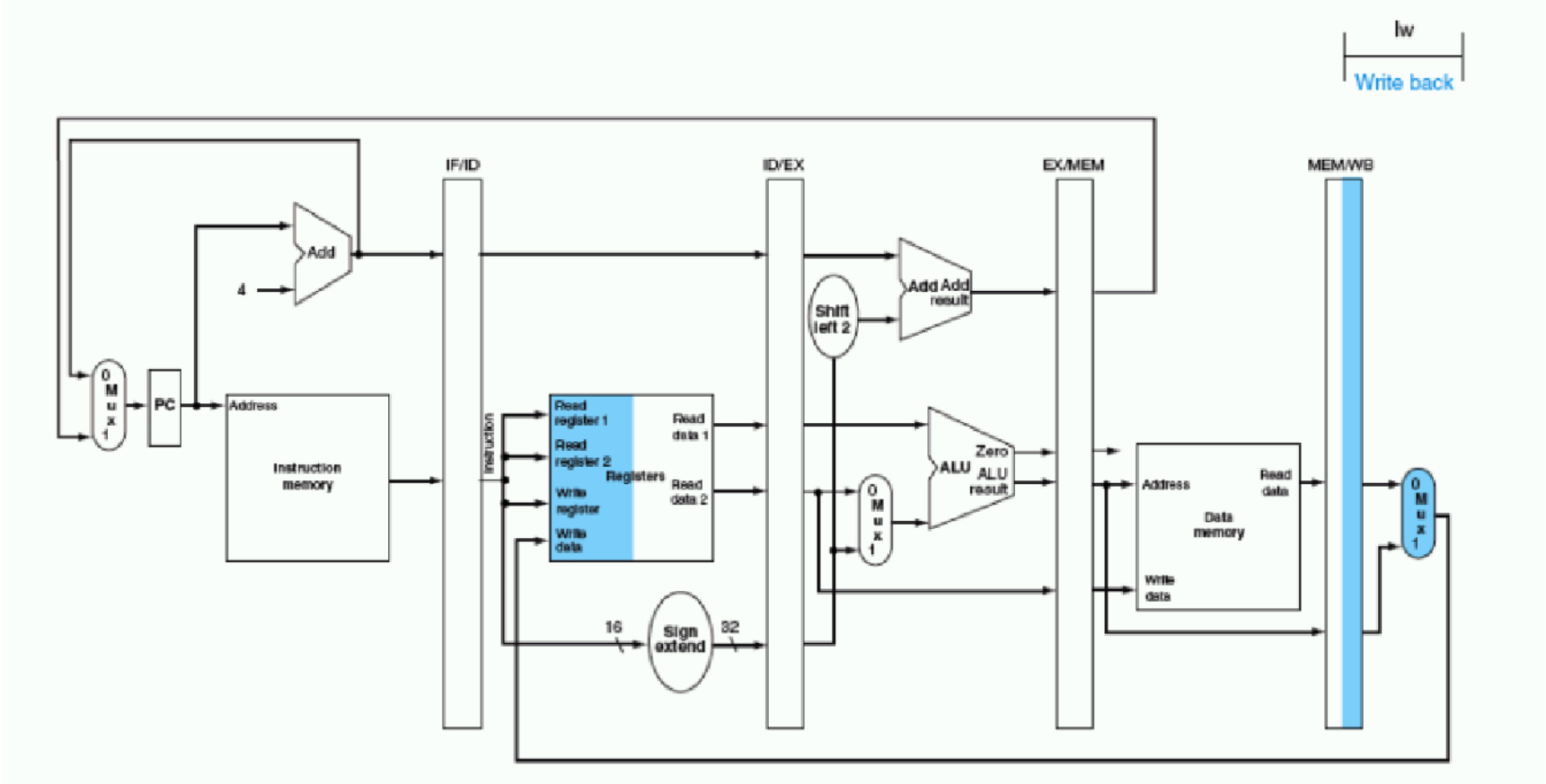
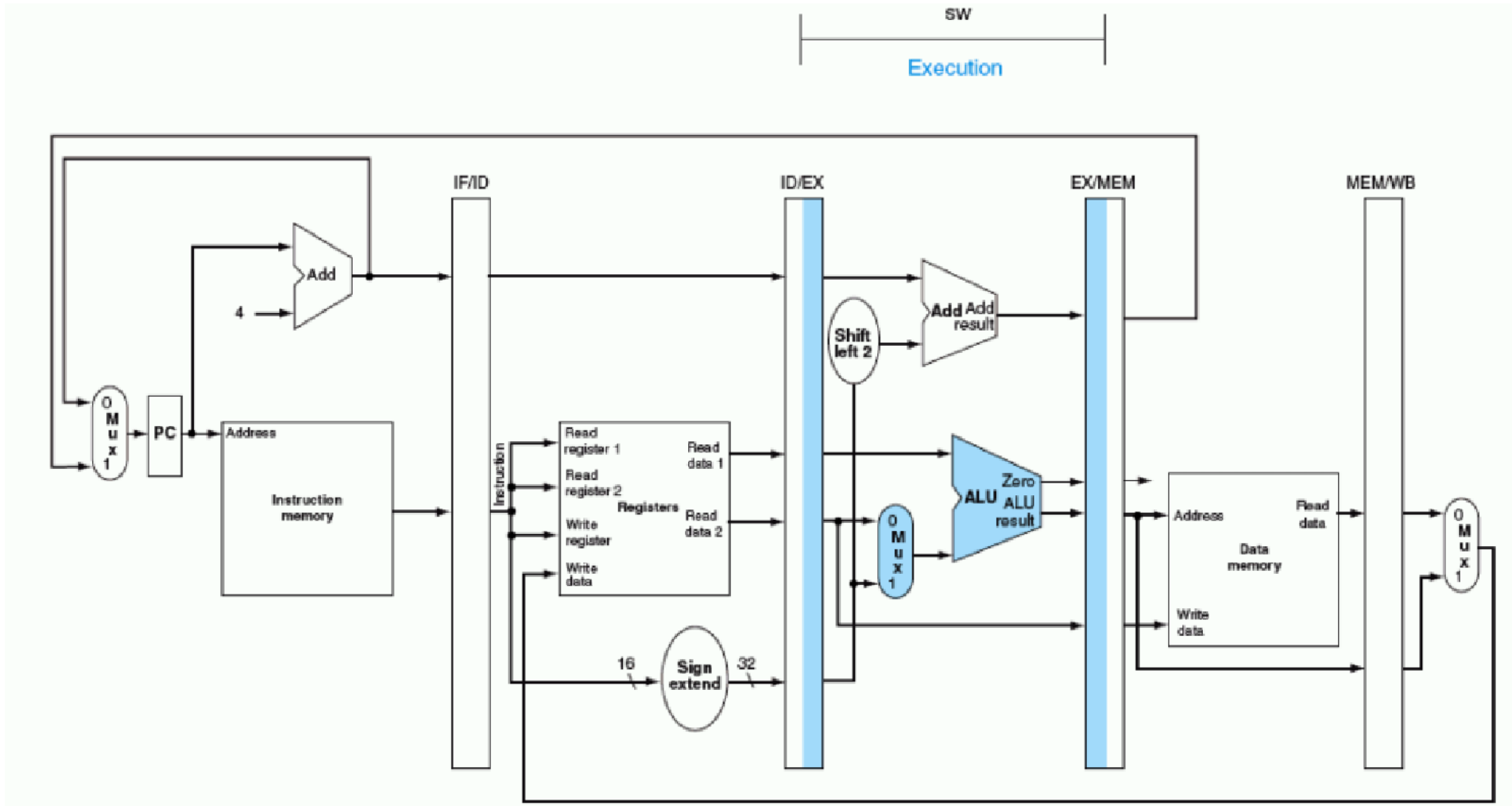
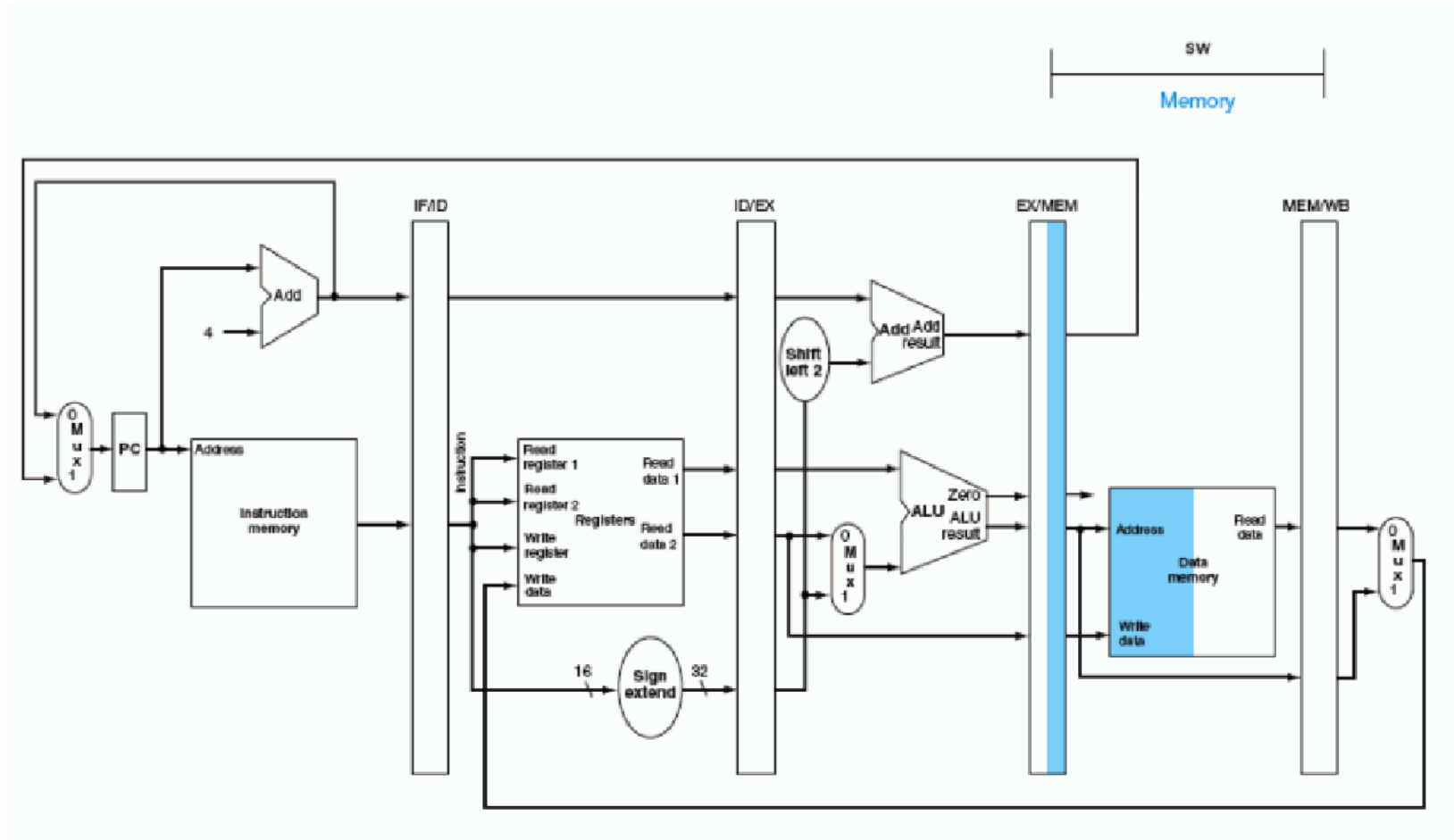


FIGURE 6.15 EX: the third pipe stage of a store instruction. Unlike the third stage of the load instruction in Figure 6.13, the second register value is loaded into the EX/MEM pipeline register to be used in the next stage. Although it wouldn't hurt to always write this second register into the EX/MEM pipeline register, we write the second register only on a store instruction to make the pipeline easier to understand.




SW
Memory
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M u x
0
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Shift left 2
Add
Add result
ALU
Zero
ALU result
16
Sign extend
32
Address
Data memory
Read data
Write data

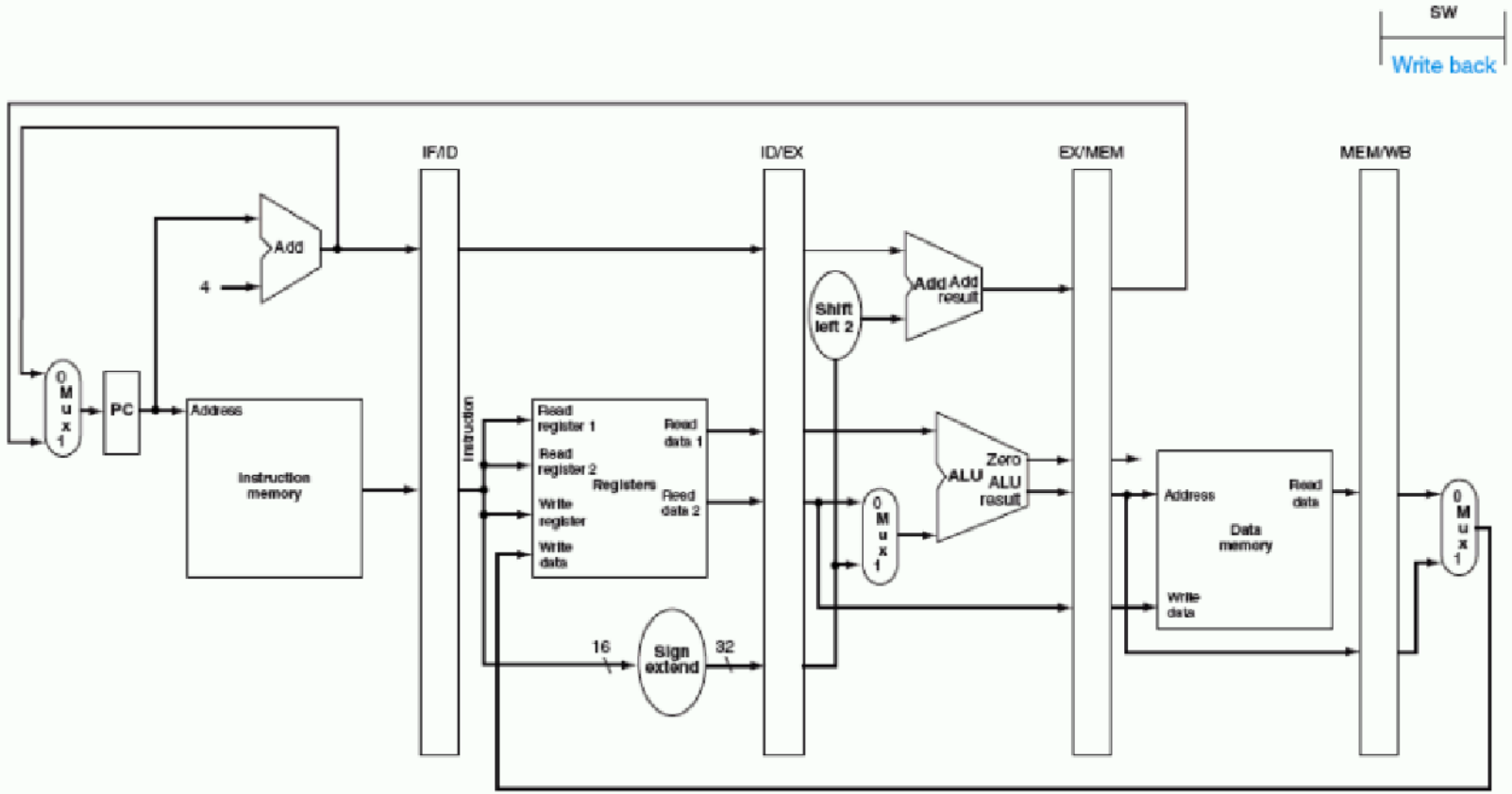


FIGURE 6.16 MEM and WB: the fourth and fifth pipe stage of a store instruction. In the fourth stage, the data is written into data memory for the store. Note that the data comes from the EX/MEM pipeline register and that nothing is changed in the MEM/WB pipeline register. Once the data is written in memory, there is nothing left for the store instruction to do, so nothing happens in stage 5.

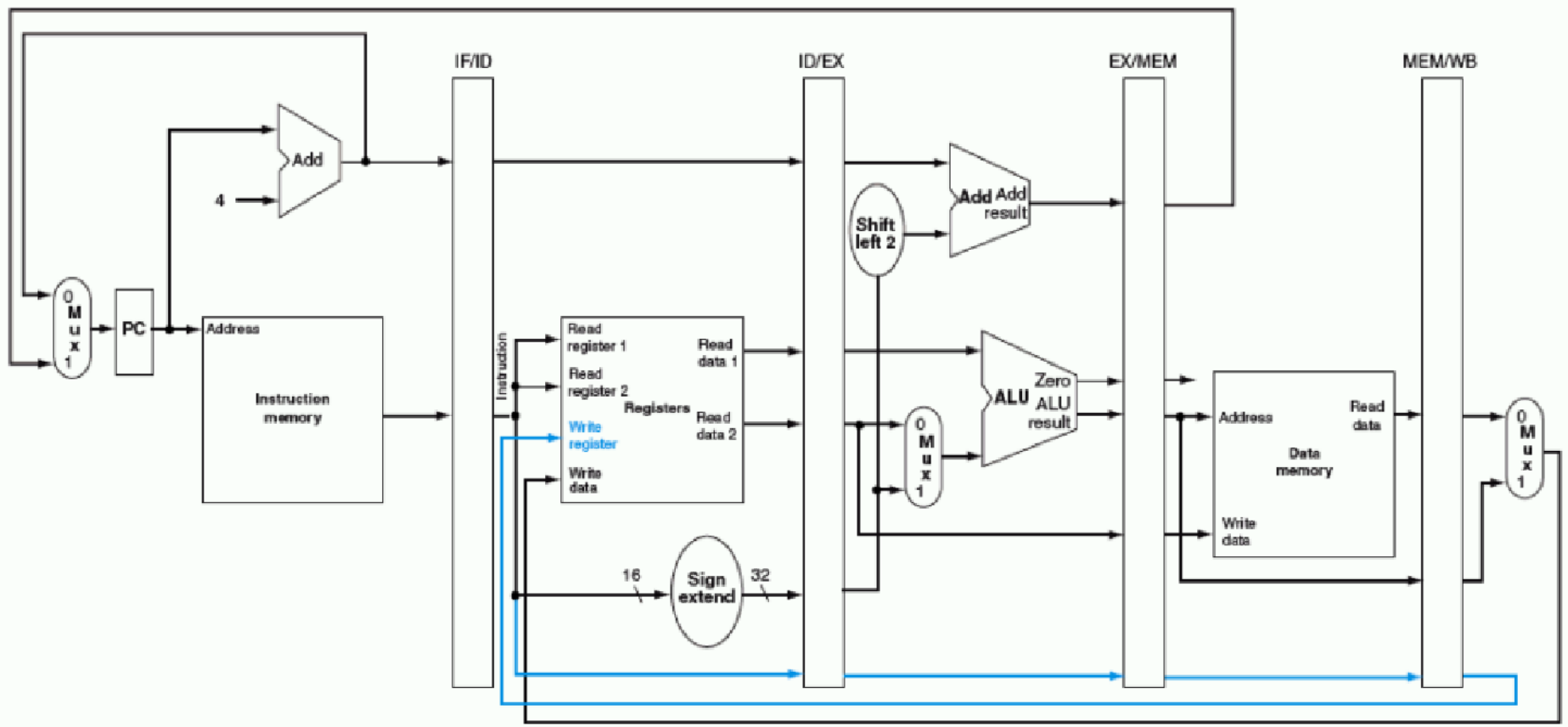


FIGURE 6.17 The corrected pipelined datapath to properly handle the load instruction. The write register number now comes from the MEM/WB pipeline register along with the data. The register number is passed from the ID pipe stage until it reaches the MEM/WB pipeline register, adding 5 more bits to the last three pipeline registers. This new path is shown in color.

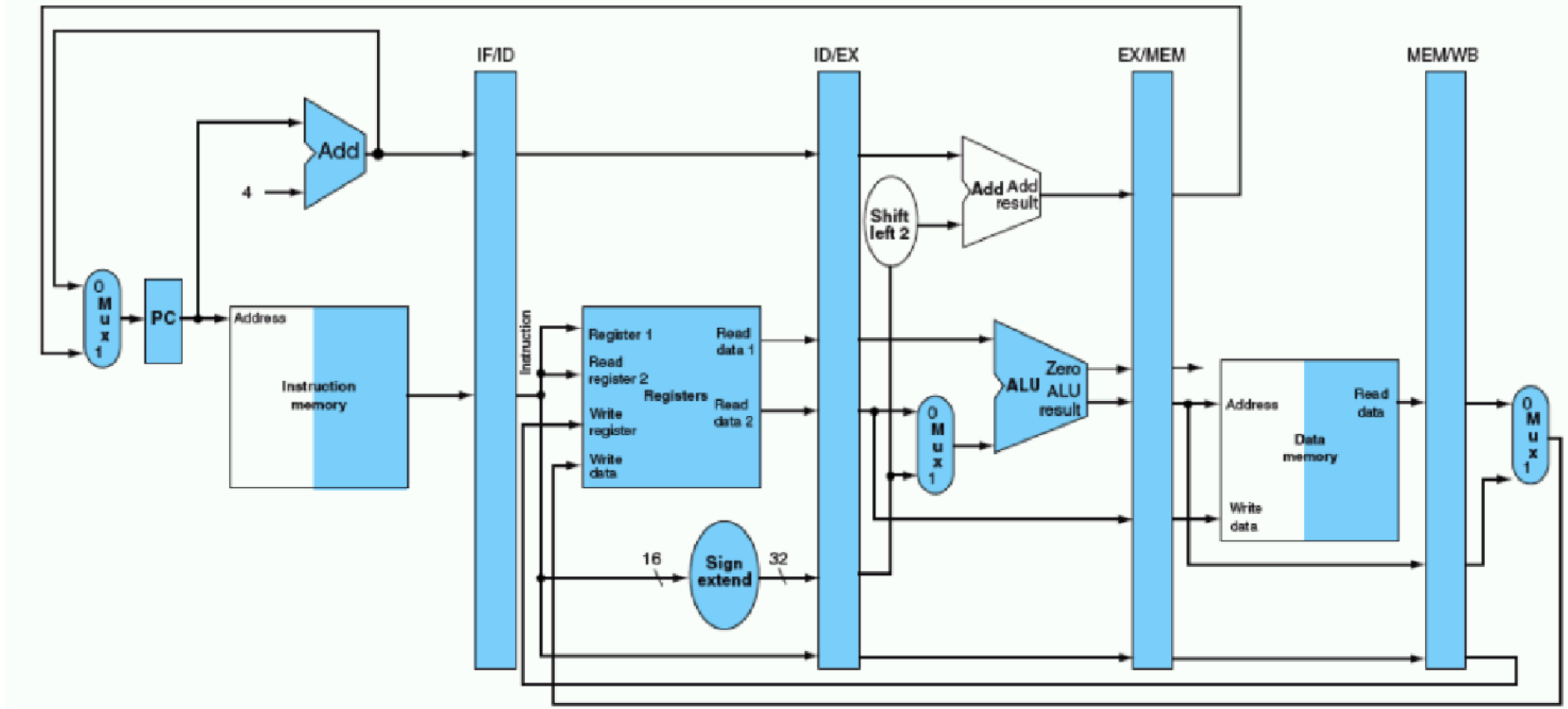
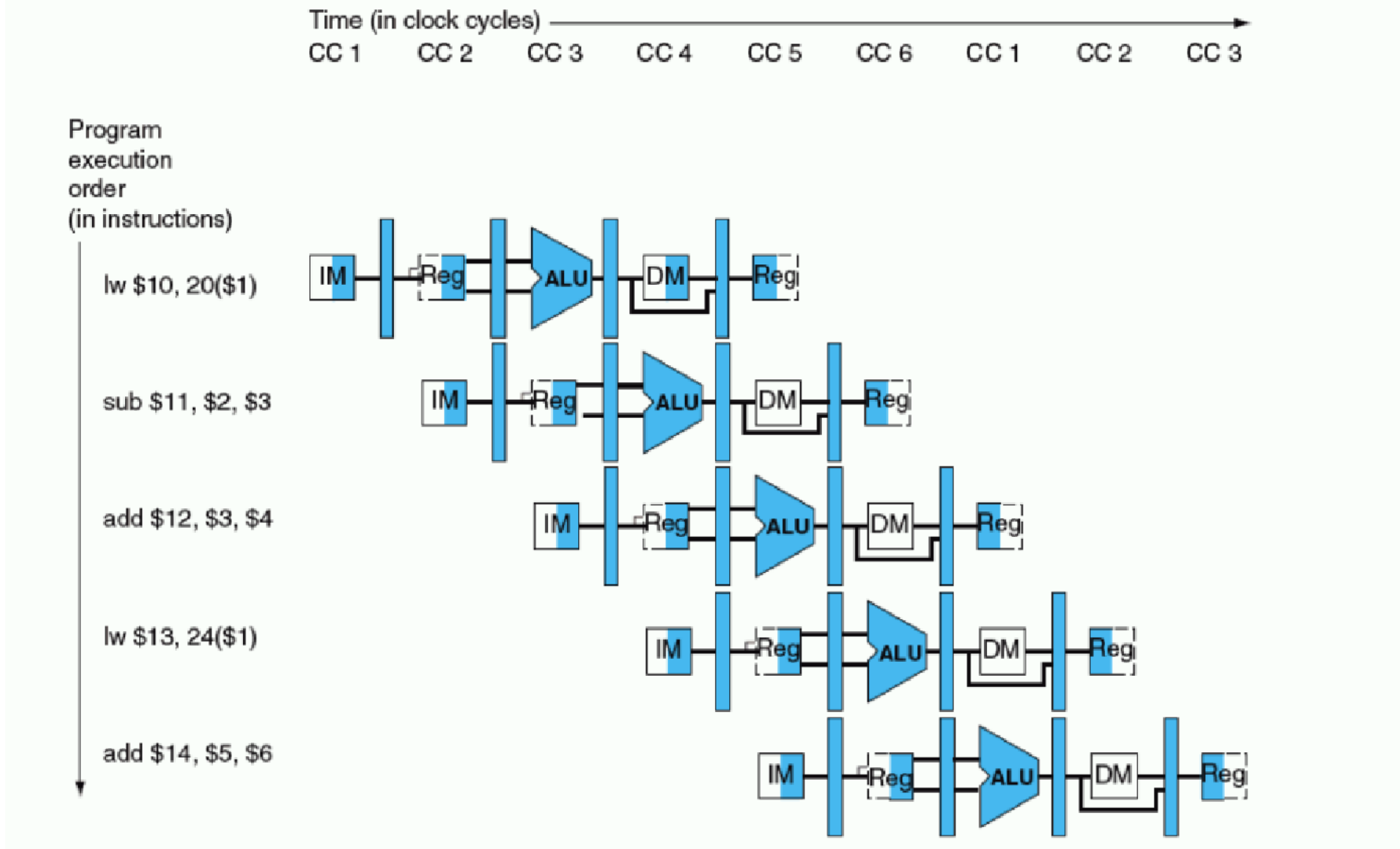


FIGURE 6.18 The portion of the datapath in Figure 6.17 that is used in all five stages of a load instruction.

```
lw      $10, 20($1)
sub     $11, $2, $3
add     $12, $3, $4
lw      $13, 24($1)
add     $14, $5, $6
```

FIGURE 6.19 Multiple-clock-cycle pipeline diagram of five instructions. This style of pipeline representation shows the complete execution of instructions in a single figure. Instructions are listed in instruction execution order from top to bottom, and clock cycles move from left to right. Unlike Figure 6.4, here we show the pipeline registers between each stage. Figure 6.20 shows the traditional way to draw this diagram.



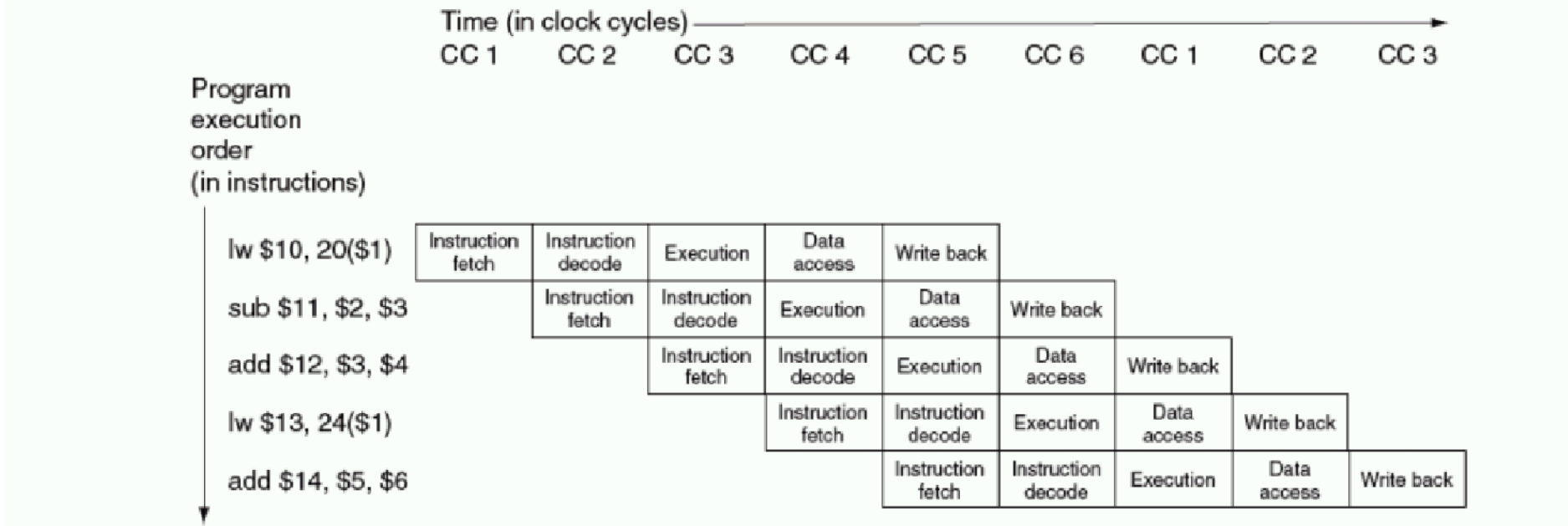
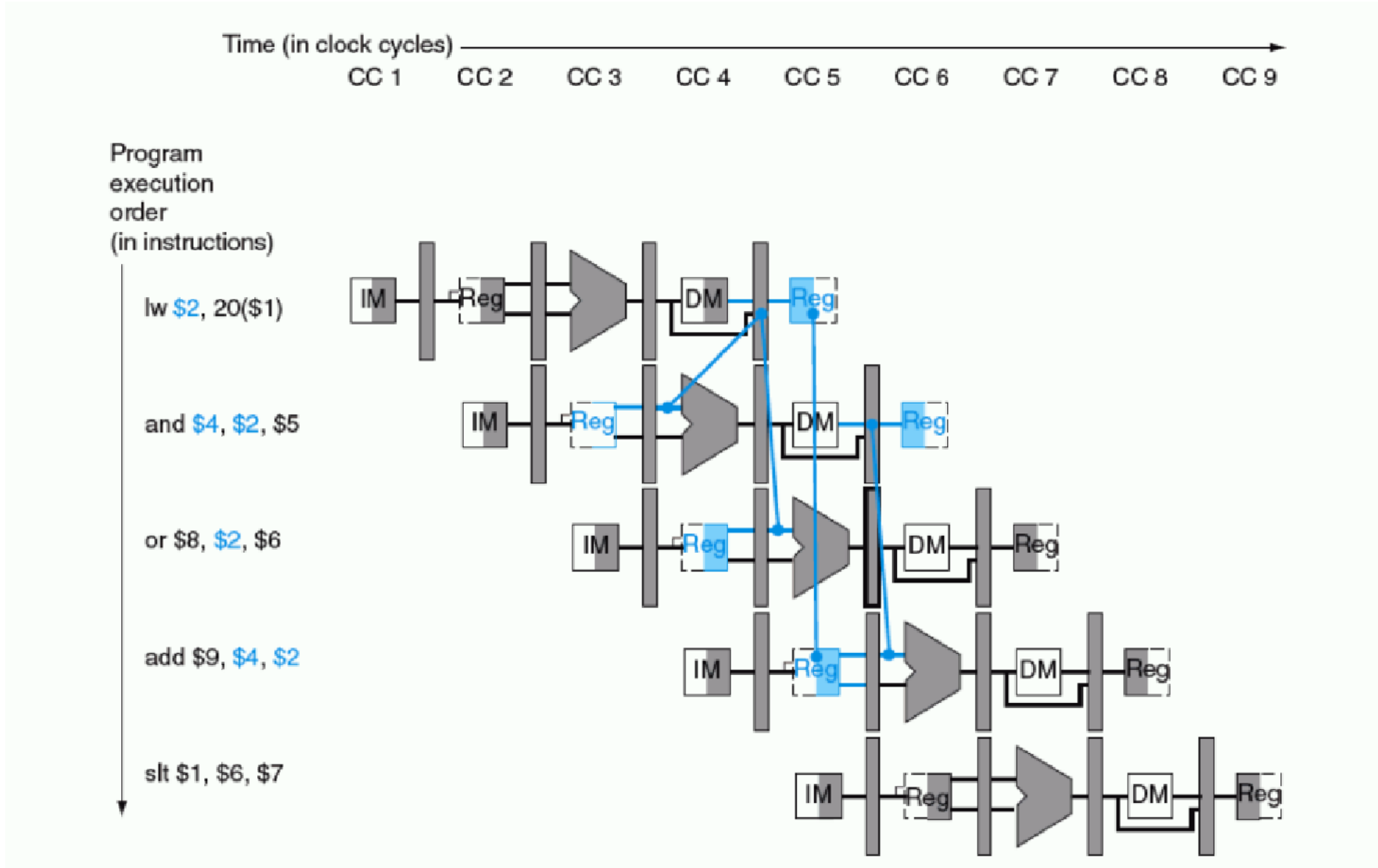
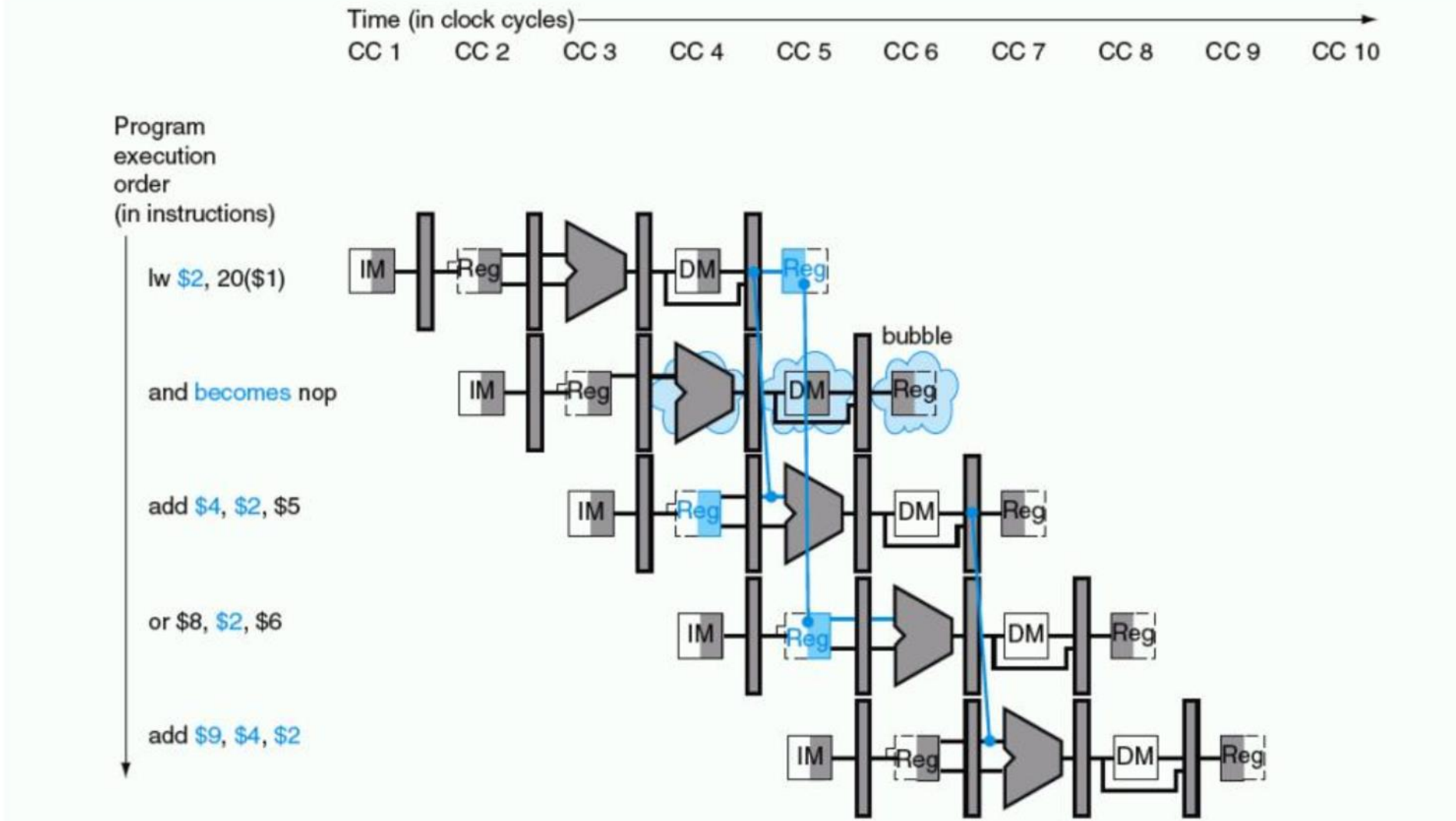
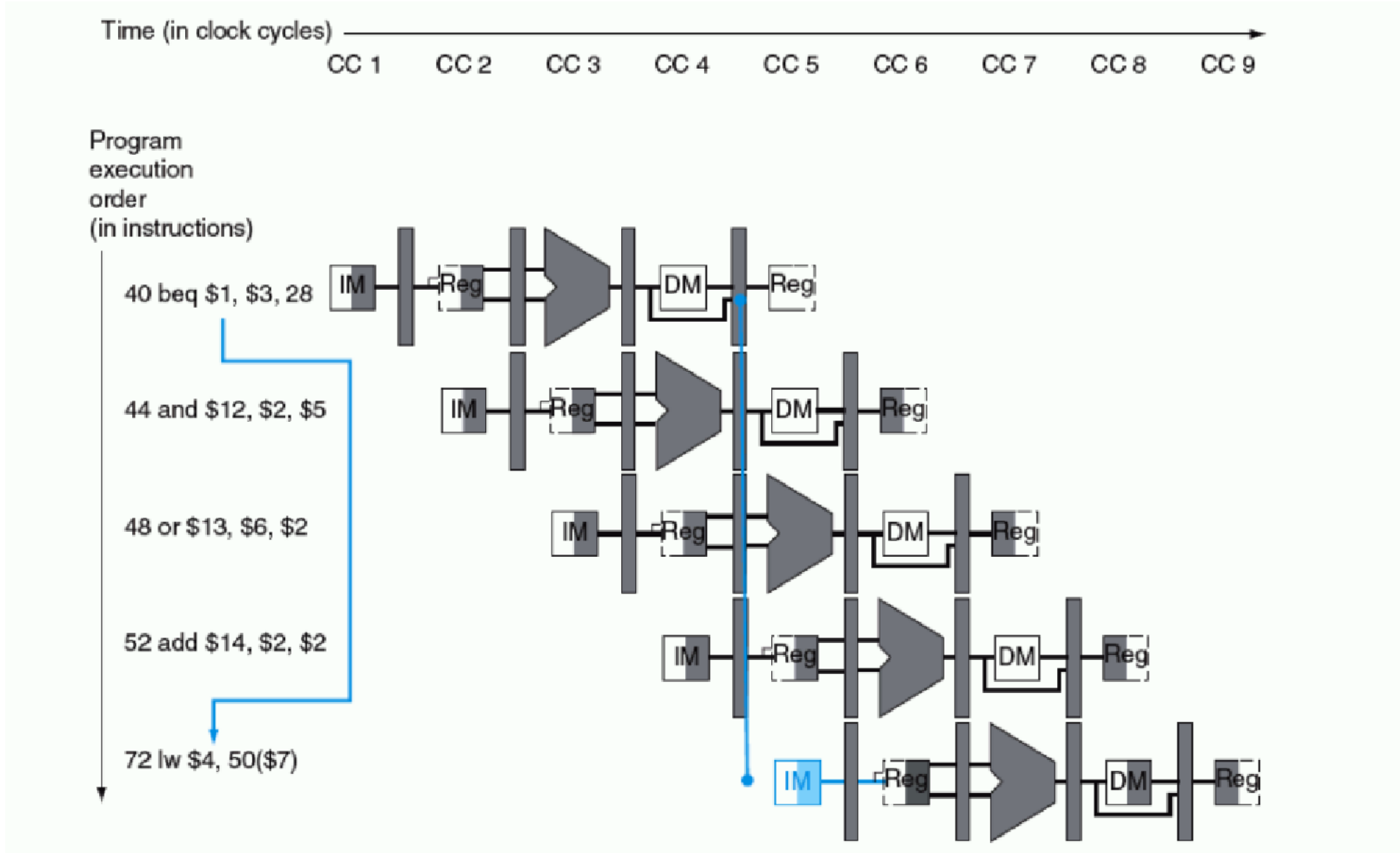


FIGURE 6.20 Traditional multiple-clock-cycle pipeline diagram of five instructions in Figure 6.19.


Time (in clock cycles)
CC 1
CC 2
CC 3
CC 4
CC 5
CC 6
CC 7
CC 8
CC 9
Program execution order (in instructions)
lw $2, 20($1)
and $4, $2, $5
or $8, $2, $6
add $9, $4, $2
slt $1, $6, $7
IM
Reg
DM


Time (in clock cycles)
CC 1
CC 2
CC 3
CC 4
CC 5
CC 6
CC 7
CC 8
CC 9
CC 10
Program execution order (in instructions)
lw $2, 20($1)
and becomes nop
add $4, $2, $5
or $8, $2, $6
add $9, $4, $2
IM
Reg
DM
bubble


Time (in clock cycles)
CC 1
CC 2
CC 3
CC 4
CC 5
CC 6
CC 7
CC 8
CC 9
Program execution order (in instructions)
40 beq $1, $3, 28
44 and $12, $2, $5
48 or $13, $6, $2
52 add $14, $2, $2
72 lw $4, 50($7)
IM
Reg
DM
Reg

- A 5 stage pipelined processor has instruction fetch (IF), instruction decode (ID), Execute (EX), Memory access (MEM) and Write Back(WB) stages. The IF, ID, MEM and WB stages take 1 clock cycle each for any instruction. The EX stage takes 1 clock cycle for ADD and SUB,3 clock cycles for MUL and 6 clock cycle for DIV respectively. Operand forwarding is used in the pipeline. What is the number of clock cycles needed to execute the following sequence of instructions?
- I_0: MUL R2,R0,R1 , I_1: DIV R5,R3,R4 , I_2: ADD R2,R5,R2 , I_3: SUB R5,R2,R6
-

- A four stage pipeline has the delays as 150,120,160 and 140ns respectively. Registers that are used between the stages have a delay of 5ns each. Assuming constant clocking rate. Calculate the total time taken to process 1000 data items on this pipeline .

- A non-pipeline system takes 60ns to process a task. The same task can be processed in a 8 segment pipeline with a clock cycle of 10ns.What is speedup ratio of the pipeline for 200 tasks and the maximum speedup that can be achieved?

-

Memory

- The topics to be discussed in this chapter aid programmers by creating the illusion of unlimited fast memory.
- **Principle of locality**

  - Temporal locality (locality in time): If an item is referenced, it will tend to be referenced again soon. If you recently brought a book to your desk to look at, you will probably need to look at it again soon.

  - Spatial locality (locality in space): If an item is referenced, items whose addresses are close by will tend to be referenced soon. For example, when

Just as accesses to books on the desk naturally exhibit locality, locality in programs arises from simple and natural program structures. For example, most programs contain loops, so instructions and data are likely to be accessed repeatedly, showing high amounts of temporal locality. Since instructions are normally accessed sequentially, programs show high spatial locality. Accesses to data also exhibit a natural spatial locality. For example, accesses to elements of an array or a record will naturally have high degrees of spatial locality.

We take advantage of the principle of locality by implementing the memory of a computer as a memory hierarchy. A memory hierarchy consists of multiple levels of memory with different speeds and sizes. The faster memories are more expensive per bit than the slower memories and thus smaller.

Memory technology	Typical access time	$ per GB in 2004
SRAM	0.5–5 ns	$4000–$10,000
DRAM	50–70 ns	$100–$200
Magnetic disk	5,000,000–20,000,000 ns	$0.50–$2